Sliding Block Cipher Version 2 and the FPMQ3 Password File: Design, Repairs, and Open Questions

Ben P. Wise

21 October 2026

Abstract

The Sliding Block Cipher (SBC) is a block cipher of the author's design, first built in 2002 [1]. It encrypts the password file of `fpwdman`, the author's password manager. A review in October 2026 found no practical way to break SBC, but it found one structural weakness in the cipher and four weaknesses in the file format and the program around it. This article explains in plain language how the cipher and the file format work and why each part is there. It then describes the weaknesses and how they were repaired in SBC version 1 and in a new file format called FPMQ2, which `fpwdman` 4.0 writes. Two further changes, a rotation whose amount depends on the data and a permutation that depends on the key, make SBC version 2 and the file format FPMQ3, which `fpwdman` 4.1 writes. The article gives pseudo-code for the cipher and the file format, and it lists the attacks and analyses that remain to be done.

Contents

1 Introduction **4**

2 How it works, and why **4**

2.1 The cipher 4

2.2 Why each part of the cipher is there 7

2.3 The password file 8

2.4 Why each part of the file is there 9

3 Weaknesses found and repaired **10**

3.1 Identical rounds and the slide attack 10

3.2 The unchecked iteration count 12

3.3 The threefold cost of key derivation 12

3.4 The unauthenticated header 13

3.5 The backup copy 13

3.6 The fixed rotation and the unkeyed permutation 13

3.7 Earlier repairs 14

4 Pseudo-code **14**

4.1 The basic SBC 14

4.2 FPWDMAN 15

5 Remaining attacks and analyses **20**

5.1 Analyses of the cipher 20

5.2 Analyses of the file format and the program 22

A Measurements 24

1 Introduction

A password manager keeps many secrets under one secret. The user remembers a single master passphrase; the program keeps every other password in a file encrypted under a key made from that passphrase. Anyone who copies the file can try to open it at leisure, on as many computers as they can afford. The design must therefore make two things hard: recovering the contents without the passphrase, and finding the passphrase by guessing.

`fpwdman` meets the first need with the Sliding Block Cipher (SBC), and the second with a standard key-derivation function. Section 2 explains both in plain language. Section 3 describes the weaknesses found in the review and how they were repaired. Section 4 gives pseudo-code for the cipher and for the file format. Section 5 lists what remains to be studied. Appendix A records the measurements on which the article relies.

Four file formats appear throughout:

- the *legacy* format, written by the original FLTK program, which is read but no longer written;
- *FPMQ1*, written by the Qt program before version 4.0, which is also read but no longer written;
- *FPMQ2*, written by version 4.0, which is also read but no longer written;
- *FPMQ3*, written by version 4.1 and later.

Opening a file in an older format and saving it converts it to FPMQ3. The cipher likewise has three versions: version 0 for the legacy format and FPMQ1, version 1 for FPMQ2, and version 2 for FPMQ3.

2 How it works, and why

2.1 The cipher

SBC treats a block of data as a ring of K words of 32 bits each. The `fpwdman` settings use $K = 32$, so a block holds 1024 bits (128 bytes). The ring has no first or last word: the word before word 0 is word $K - 1$. A classic Feistel cipher (such as the US Data Encryption Standard, Blowfish, and many more) uses the same ring structure with $K = 2$. As there are only two elements in such a ring, they are conventionally referred to as the left and right blocks.

Encryption proceeds one *step* at a time. A step changes exactly one word, using the word just before it in the ring and one 32-bit *subkey*. Step 0 changes word 0, step 1 changes word 1, and so on around the ring. One pass around the ring is one *round*. `fpwdman` uses $R = 24$ rounds, that is $24 \times 32 = 768$ steps.

The change made at each step is

$$\begin{aligned} q(x) &= (x+1)(2x+1) \bmod 2^{32}, & (2.1)\\ x_p &\leftarrow x_p + \big(q(x_{p-1} \oplus s_i) \lll 7\big) \bmod 2^{32}, & (2.2) \end{aligned}$$

where x_p is the word being changed, x_{p-1} the word before it, s_i the subkey for step i, $\oplus$ is bitwise exclusive-or, and $\lll 7$ rotates the 32 bits left by seven places. The function q (called `qTransform` in the code) is a quadratic that maps each 32-bit number to a different 32-bit number. It was adapted from the cipher RC6, and the added constant 1 removes the fixed points that RC6's version has.

The author adapted the quadratic transformation of RC6 to derive the general form of q as follows.

$$q(x) = (x+a)(nx+c) \bmod 2^m, \tag{2.3}$$

To ensure that $q(x) = q(y)$ if and only if $x = y$, we require that n is even positive, and c be odd positive. We prove the bilateral implication first by assuming $x \neq y$ and showing that $q(x) \neq q(y)$ follows.

$$\begin{aligned} q(x) - q(y) &= \big(nx^2 + (an+c)x + ac\big) - \big(ny^2 + (an+c)y + ac\big) & (2.4)\\ &= \big(n(x+y) + (an+c)\big)\,(x-y) & (2.5) \end{aligned}$$

Here x and y are words, numbers from 0 to $2^m - 1$, and the right-hand side is computed in ordinary integers. Without loss of generality, assume that $x > y$. Write u for the first factor of the right-hand side:

$$u = n(x+y) + an + c. \tag{2.6}$$

The number u is odd, because $n(x+y)$ and an are even (as n is even) and c is odd.

Suppose, to the contrary, that $q(x) = q(y)$, that is, that $q(x) - q(y)$ is a multiple of 2^m. Then 2^m divides the product $u\,(x-y)$. Since u is odd, it contains no factor of 2, so all m of the factors of 2 must lie in $x - y$:

$$2^m \mid u\,(x-y) \;\text{ and }\; u \text{ odd} \implies 2^m \mid (x-y). \tag{2.7}$$

But x and y are different numbers from 0 to $2^m - 1$, so $0 < x - y < 2^m$, and no multiple of 2^m lies strictly between 0 and 2^m. The supposition is therefore false, and $q(x) \neq q(y)$.

The converse, that $x = y$ gives $q(x) = q(y)$, is immediate. Hence $q(x) = q(y)$ if and only if $x = y$: q maps the 2^m possible words to 2^m different words, so it merely reorders them, and it can be undone.

To ensure that q has no fixed points , we require that a is odd positive. The proof is particularly simple. Expanding the product and subtracting x gives

$$\begin{aligned} q(x) - x &= \left(nx^2 + (an + c)\, x + ac\right) - x && (2.8) \\ &= nx^2 + (an + c - 1)\, x + ac. && (2.9) \end{aligned}$$

The first term is even, because n is even. The second is even, because an is even and $c - 1$ is even, c being odd. The last term, ac, is odd, because a and c are both odd. The right-hand side is therefore odd for every x. But $q(x) = x$ would require $q(x) - x$ to be a multiple of 2^m, and so even. Hence there can be no fixed point.

The choice of n matters in one further way: it should be twice an odd number. Expanding the product gives

$$q(x) = nx^2 + (an + c)\, x + ac \bmod 2^m. \quad (2.10)$$

Let 2^k be the largest power of two that divides n. The squared term nx^2 is then zero modulo 2^k, so the lowest k bits of $q(x)$ come from the other two terms alone:

$$q(x) \equiv (an + c)\, x + ac \pmod{2^k}. \quad (2.11)$$

In those k bits, q only multiplies its input by an odd number and adds a constant, which is close to linear; the nonlinear squared term begins only at bit k. It begins as low as it can, at bit 1, when $k = 1$, that is, when n is twice an odd number. The present choice $n = 2$ meets this. A multiple of four does not: with $(a, n, c) = (3, 8, 17)$, for example, $k = 3$, and the lowest three bits of $q(x)$ are simply those of $x + 3$. Whether the odd factor is prime does not matter; only its oddness does. (Four is twice a prime, and fails.) A larger $n = 2u$, with u odd and with many of its bits set, would still satisfy the rule and might spread each input bit into more of the higher output bits in a single application; whether it actually does would have to be measured.

The final set of conditions are that a is odd, n is twice an odd number, c be odd, and all three are positive.

In version 2 each step ends with a second rotation, of the word being changed, by an amount taken from the data. With t the output of q and r its five highest bits, a number from 0 to 31,

the step is

$$t = q(x_{p-1} \oplus s_i), \tag{2.12}$$
$$r = \lfloor t/2^{27} \rfloor, \tag{2.13}$$
$$x_p \leftarrow \left(x_p + (t \lll 7) \bmod 2^{32}\right) \lll r. \tag{2.14}$$

Versions 0 and 1 stop at step (2.2), which is (2.14) with $r = 0$.

Every 64 steps (two rounds) the words of the ring are rearranged by a permutation called `perm1`. It does not change any word; it only moves words to new places. The new order depends on the data: `perm1` combines all the words into one 32-bit number, by applying q to each and taking the exclusive-or of the results, and uses that number to choose the rearrangement. It does so in eight moves. At each it adds a different part of the number to every word's position, modulo K, and then folds the positions over: a word in the first half moves to twice its position, and a word in the second half moves, in reverse order, to an odd position. Version 2 makes two changes. It combines the number, by exclusive-or, with one more secret subkey for each permutation, so that the rearrangement can no longer be computed without the key. And it rotates the number by five places between the moves, instead of by seven, so that the eight moves use all 32 of its bits; with seven, two of the bits are never used.

A file is longer than one block, so the blocks are chained: each block is combined, by exclusive-or, with the encrypted block before it, and then encrypted. This is the standard cipher-block-chaining (CBC) mode. Decryption runs every step backward, from the last step to the first, subtracting where encryption added.

2.2 Why each part of the cipher is there

The step can always be undone.
Step (2.2) changes x_p by an amount that depends only on x_{p-1} and s_i, and x_{p-1} is not changed by that step. Whoever holds the key can therefore compute the same amount again and subtract it. This is the same idea as the Feistel ciphers, generalized from two words to a ring of any size, which is what lets SBC handle blocks of any length.

The step mixes bits nonlinearly.
Encryption that used only exclusive-or and addition could be undone by solving linear equations. The product in q and the mixing of exclusive-or with addition prevent that.

The rotation moves the high bits down.
In q, each output bit depends only on the input bits at the same place and below. Without

the rotation, a change in a high bit could never reach the low bits. Rotating by seven places brings high bits to the bottom, so over many steps every bit influences every other.

The second rotation depends on the data (version 2).
The amount r comes from the five highest bits of q, which depend on every input bit. Where each bit of the changed word lands therefore depends on the whole of x_{p-1} and the subkey, so within a single step the low bits of the result depend on the high bits of the input. The fixed rotation by seven is kept, so that the high bits of q still reach the bottom in the one step in 32 where r is zero. The step can still be undone, because r, like t, depends only on x_{p-1} and s_i. The rotation is computed by a single processor instruction whose time does not depend on the amount, so the time a step takes reveals nothing about r.

The ring spreads a change to every word.
A change to word p alters the next step's change to word $p + 1$, which alters word $p + 2$, and so on. Measured on random data, one round changes about a quarter of the 1024 bits of a block, and two rounds change half of them, which is what a random result would do (Appendix A).

The permutation moves words apart.
The step relates only neighbouring words. Moving words to new places between rounds makes words that were far apart into neighbours and vice versa.

The permutation can be undone, and in version 2 it is secret.
The number that chooses the rearrangement does not depend on the order of the words, so decryption can compute it again from the rearranged block. In version 2 it also includes a subkey, so the rearrangement cannot be computed without the key. Because the number depends on every word, the two blocks of a pair that differ almost always receive different rearrangements, which breaks the pattern of differences that a differential attack follows.

Many rounds give a margin.
Most attacks on block ciphers grow harder with every round. Twenty-four rounds are far more than are needed to mix the bits, which leaves room against attacks not yet known.

Every step has its own subkey (versions 1 and 2).
Section 3.1 explains why this matters.

2.3 The password file

Saving the password file in FPMQ3 takes the following stages. (FPMQ2 differs only in using SBC version 1 and its own labels in stage 3.)

1. Choose 16 random bytes, the *salt*.
2. Run PBKDF2 [2] on the passphrase and salt, with 600,000 repetitions of HMAC-SHA-256 [3, 4], to obtain a 32-byte *master key*.
3. Expand the master key with HKDF [5] into two independent keys: 64 bytes for the cipher and 32 bytes for the authentication code.
4. Place 16 more random bytes, the *nonce*, before the list of entries, followed by its length; pad the whole to a multiple of 128 bytes.
5. Encrypt it with SBC version 2 in CBC mode.
6. Compute an authentication code (HMAC-SHA-256) over the file's header and the encrypted text, and write the header, the code, and the encrypted text.

Opening the file repeats stages 2 and 3, recomputes the authentication code, and decrypts only if it matches.

2.4 Why each part of the file is there

The 600,000 repetitions make guessing slow.
The user pays the cost once when opening the file, about a second. An attacker pays it for every passphrase tried. This cost is the main protection for a stolen file, and it must not be reduced to make the program feel faster.

The salt prevents precomputation.
Because every file has its own salt, an attacker cannot compute a table of common passphrases and their keys once and use it against every file. Each guess is good for one file only. The salt need not be secret, and it cannot be: the program needs it to make the key, and the key is needed to decrypt anything. It is therefore written in the clear, as in every standard format.

The nonce makes every encryption different.
The random bytes at the front of the plaintext change the whole encrypted file, because CBC carries the change from the first block into every later one. Since the salt is also new at every save, the key changes too; the nonce is a second safeguard.

The authentication code detects a wrong passphrase and any tampering.
The code is checked before anything is decrypted. A wrong passphrase gives a wrong key, and therefore a code that does not match. A file changed by anyone without the passphrase also fails the check. Checking the code before decrypting ("encrypt-then-MAC") is the order that is known to be safe [6].

The cipher and authentication keys are separate.
Using one key for two purposes invites unforeseen interactions between them.

3 Weaknesses found and repaired

Table 1 summarizes the weaknesses. The subsections explain each.

Weakness	Effect	Repair
Every round of SBC the same	Slide attacks, whose cost does not depend on the number of rounds	SBC version 1: a different subkey for every step, made with HKDF
Iteration count read from the file without a check	A crafted file could freeze the program for about 40 minutes	Only 600,000 is accepted, checked before any key is made
Three PBKDF2 runs per open	The user paid three times what an attacker pays	FPMQ2: one PBKDF2 run, expanded with HKDF
Header outside the authentication code	Header could be changed without detection	FPMQ2: the code covers the header
A backup copy kept at every save	The previous file survived in its old format and under its old passphrase	Version 4.0 keeps no backup
A fixed rotation of seven places	Low output bits depend on high input bits only after several steps; the step keeps the regular algebra of a quadratic	SBC version 2: a further rotation by the five high bits of q
A permutation with no key, reading 30 of its 32 bits	Anyone who knows the state can compute the rearrangement	SBC version 2: a subkey for each permutation, and all 32 bits read

Table 1: Weaknesses found in the October 2026 review and their repairs.

3.1 Identical rounds and the slide attack

In version 0 of SBC, step i used the subkey $s_i = q(k_{i \bmod 16})$, where $k_0, \ldots, k_{15}$ are the 16 words of the key. The subkeys therefore repeat every 16 steps, the word positions repeat every 32, and

`perm1`, which uses no key, runs every 64. Between one permutation and the next, the cipher applied exactly the same function. A test confirmed that the state after each permutation is that same function applied to the state after the one before (Appendix A).

A cipher built of identical keyed rounds is open to the *slide attack* of Biryukov and Wagner [7, 8]. Let F be one round under a fixed key, and let the cipher be $E = F^r$, that is F applied r times. Then

$$E(F(P)) = F^{r+1}(P) \tag{3.1}$$
$$= F(E(P)). \tag{3.2}$$

Suppose an attacker holds two plaintexts P and P' with $P' = F(P)$, a *slid pair*. By (3.2) their ciphertexts satisfy $C' = F(C)$. The attacker then has two inputs and outputs of a single round, (P, P') and (C, C'). A single round is far weaker than the whole cipher, so the key can be sought from one round instead of 24.

The attacker cannot compute F, and so cannot make a slid pair on purpose, but must find one by chance among many known plaintexts and ciphertexts. By the birthday argument this takes about $2^{n/2}$ pairs for an n-bit block, and the number does not depend on r. Adding rounds does not raise it.

For SBC, with $n = 1024$, the cost is about 2^{512} known blocks under one key, which no one can collect; `fpwdman`, besides, uses a new key at every save. The weakness is therefore one of principle, not of practice. It is still a design defect, because it bypasses the number of rounds, which is the cipher's main margin of safety.

The repair. Version 1 makes a separate subkey for every step from the key, by HKDF:

$$(s_0, s_1, \ldots, s_{RK-1}) = \mathrm{HKDF}\big(\mathit{key},\ \mathit{salt} = \text{empty},\ \mathit{info} = \text{“SBC v1 subkeys”} \,\|\, K \,\|\, R,\ L = 4RK \text{ bytes}\big), \tag{3.3}$$

where $\|$ joins byte strings and K and R are written as four-byte numbers. Each run of four output bytes is one subkey. Nothing else in the cipher changes. No two rounds now use the same subkeys, and there is no simple rule that links the subkeys of one key to those of another. Writing K and R into the label keeps the schedules of different shapes unrelated: without them, the 24-round schedule would be the start of the 25-round one.

A repair that does not work. An obvious alternative is to add a different constant at each step, for example

$$s_i = q\big(k_{i \bmod 16} + (i+1)\varphi\big) \bmod 2^{32}, \tag{3.4}$$

$$\varphi = 9\mathrm{E}3779\mathrm{B}9_{16}. \tag{3.5}$$

Every step then has its own subkey, but the subkeys are related in a simple way. Take a second key k' whose every word is larger by 64φ. Step i under k' then uses exactly the subkey of step $i+64$ under k. Sixty-four steps is a whole number of rounds and also matches the permutation schedule, so the two keys give the same cipher shifted by 64 steps, and the slide attack returns for pairs of related keys. Any constant that grows regularly with the step number has this flaw. HKDF has no such regularity. A milder alternative that keeps the key word in every step is

$$s_i = q\big(k_{i \bmod N} \oplus c_i\big), \tag{3.6}$$

with c_i the first 32 bits of SHA-256 of "SBC-RC" and i. It also defeats the slide attack, but it leaves the subkeys simply related to the key, so version 1 uses HKDF.

3.2 The unchecked iteration count

The header of an FPMQ1 file states the number of PBKDF2 repetitions, and the program used that number as written, checking only that it was positive. The number cannot be authenticated before it is used, because the authentication key itself comes out of PBKDF2. A crafted file giving 2,147,483,647 repetitions would have made the program compute about 6.4 billion HMACs, roughly 40 minutes in which the window would not respond. Nothing secret would leak, so the defect was a nuisance rather than a break. Both formats now accept only 600,000, and the check is made before any key is derived.

3.3 The threefold cost of key derivation

PBKDF2 produces 32 bytes for each full run of its repetitions. FPMQ1 asked for 96 bytes, so every open ran PBKDF2 three times: 1.8 million HMACs. But the authentication key was the third 32-byte block by itself, so an attacker testing a guess needed to compute only that block, 600,000 HMACs. The extra 1.2 million gave the user delay and no protection. FPMQ2 runs PBKDF2 once for 32 bytes and expands them with HKDF, which costs two HMACs. Saving and opening a file now takes about a third as long as before (Appendix A), with the attacker's cost unchanged.

3.4 The unauthenticated header

FPMQ1 computed its authentication code over the encrypted text only. Changing the salt or the repetition count would still have failed, because those change the key, but the version name and the other header text were not protected. FPMQ2 computes the code over both header lines and the encrypted text, and it rejects any header that differs from what the program writes. In particular, an FPMQ2 file relabelled as FPMQ1 is refused, so it cannot be steered onto the older reading path. FPMQ3 does the same, and since each format derives its keys with its own labels, an FPMQ3 file relabelled as FPMQ2 or FPMQ1 is refused too.

3.5 The backup copy

Before version 4.0, every save first copied the existing file to a backup with the extension `.bak`. The backup kept the previous file exactly, in its old format and under its old passphrase. Two cases made that a weakness. After a legacy file was converted, the backup was the legacy file itself, which allows fast guessing of a passphrase that also opens the new file. After a change of master passphrase, the backup could still be opened with the old one, which defeats the purpose of changing it. Version 4.0 keeps no backup. A failed save is still harmless: the program writes the new file under a temporary name and replaces the old one only when the new one is complete, so an interrupted save leaves the old file as it was.

3.6 The fixed rotation and the unkeyed permutation

These two weaknesses were not found by an attack but by reasoning about where an attack would begin; Section 5 (items 5 and 7) gives the reasoning. Version 2 of SBC repairs both, and FPMQ3 is the file format that uses it.

The rotation. In versions 0 and 1, each output bit of q depends only on the input bits at its own place and below, and only the fixed rotation by seven places carries information downward, over several steps. The step also keeps the regular algebra of a quadratic. Version 2 adds the rotation of steps (2.12) to (2.14): the changed word is rotated by the five highest bits of q. A differential or linear attack must now predict one of 32 rotation amounts at every step, and a rotation by a varying amount is not a polynomial, so the step no longer has a quadratic's regular algebra. The rotation is computed without any branch on its amount, so its timing reveals nothing.

The permutation. In versions 0 and 1, `perm1` uses no key: anyone who knows the state at that point can compute the rearrangement. It also reads only 30 of the 32 bits of the number that

drives it. Version 2 combines that number with a secret subkey for each permutation, drawn from the same HKDF expansion as the step subkeys, and rotates the number by five places between moves, so that all 32 bits are read. Decryption is unchanged in form: it computes the number from the rearranged block, combines it with the same subkey, and undoes the moves in reverse order.

Version 2 needs 11 more subkeys than version 1, one for each permutation, and a few more instructions per step. Its cost is negligible beside that of PBKDF2.

3.7 Earlier repairs

For completeness: the legacy format of the FLTK program had no key-derivation function. The passphrase was scrambled into the key by a reversible function, with no salt and no delay, and a wrong guess could be recognized after decrypting a single block. Guessing was therefore millions of times cheaper than for FPMQ1, FPMQ2 or FPMQ3. The Qt program corrected this with FPMQ1 before the present review, and it now only reads the legacy format.

4 Pseudo-code

Throughout, words are 32 bits; $+$ and $-$ are modulo 2^{32}; $\oplus$ is exclusive-or; $\lll$ is left rotation. The cipher's parameters are the block size K (32), the key size N (16 words), the rounds R (24), and the permutation interval $M = 2K$ (64).

4.1 The basic SBC

Algorithms 1 to 4 give the cipher: its primitive functions, its three key schedules, the encryption and decryption of one block, and the chaining of blocks. The version v (0, 1 or 2) selects the lines that differ. Three points need comment. First, in PERM1 the value σ does not depend on the order of the words, so decryption can compute it from the permuted block, and so undo the permutation. Second, in version 2 the rotation amount r depends only on x_{p-1} and s_i, which the step leaves unchanged, so decryption can compute it too. Third, the chaining starts from zero rather than from a random value, but the first 16 bytes of every plaintext are random (Algorithm 5), which serves the same purpose.

Algorithm 1 The primitive functions

function Q(x)
    return $(x+1)(2x+1)$ ▷ modulo 2^{32}
end function
function FOLD(j) ▷ stretch and fold of the indices $0 \ldots K-1$
    if $2j \le K-1$ **then**
        return $2j$
    else
        return $2(K-j)-1$
    end if
end function
function PERM1(x, *forward*, κ, ρ) ▷ versions 0, 1: $\kappa = 0$, $\rho = 7$; version 2: $\kappa = \pi_m$, $\rho = 5$
    $\sigma \leftarrow \kappa \oplus q(x_0) \oplus q(x_1) \oplus \cdots \oplus q(x_{K-1})$ ▷ does not depend on the order
    for $t \leftarrow 0$ **to** 7 **do**
        $\sigma_t \leftarrow \sigma \lll \rho t$
    end for
    if *forward* **then**
        for $t \leftarrow 0$ **to** 7 **do**
            for every j: $y_{\text{FOLD}((j+\sigma_t) \bmod K)} \leftarrow x_j$; then $x \leftarrow y$
        end for
    else ▷ undo the eight moves in reverse order
        for $t \leftarrow 7$ **down to** 0 **do**
            for every j: $y_j \leftarrow x_{\text{FOLD}((j+\sigma_t) \bmod K)}$; then $x \leftarrow y$
        end for
    end if
    return x
end function

4.2 FPWDMAN

Algorithm 5 saves the password file and Algorithm 6 opens it. A legacy file is recognized by its first line, which holds two numbers, and is decoded by a separate reader that reproduces the FLTK program exactly. Saving any file, whatever its format, calls SAVE.

Algorithm 2 Key setup

function KeySetupV0($k_0, \ldots, k_{N-1}$) ▷ legacy and FPMQ1 only
  for $i \leftarrow 0$ **to** $RK - 1$ **do**
    $s_i \leftarrow q(k_{i \bmod N})$ ▷ repeats every N steps
  end for
  return s
end function
function KeySetupV1(key) ▷ any non-empty byte string
  $info \leftarrow$ "SBC v1 subkeys" $\|$ BE32(K) $\|$ BE32(R)
  $b \leftarrow$ HKDF-SHA-256(key, empty salt, $info$, $4RK$)
  for $i \leftarrow 0$ **to** $RK - 1$ **do**
    $s_i \leftarrow$ bytes $4i \ldots 4i + 3$ of b, read most significant first
  end for
  return s ▷ $RK \leq 2040$, the limit of HKDF
end function
function KeySetupV2(key) ▷ FPMQ3; any non-empty byte string
  $P \leftarrow \lfloor (RK - 1)/M \rfloor$ ▷ permutations per block: 11
  $info \leftarrow$ "SBC v2 subkeys" $\|$ BE32(K) $\|$ BE32(R)
  $b \leftarrow$ HKDF-SHA-256(key, empty salt, $info$, $4(RK + P)$)
  $w_0, \ldots, w_{RK+P-1} \leftarrow$ the words of b, read most significant first
  $s_i \leftarrow w_i$ for $0 \leq i < RK$; $\pi_m \leftarrow w_{RK+m}$ for $0 \leq m < P$
  return s, π ▷ $RK + P \leq 2040$
end function

Algorithm 3 Encrypting and decrypting one block

function EncipherBlock($x_0, \ldots, x_{K-1}$; s, π, v)
  for $i \leftarrow 0$ **to** $RK - 1$ **do**
    $p \leftarrow i \bmod K$
    $t \leftarrow q(x_{(p-1) \bmod K} \oplus s_i)$
    $x_p \leftarrow x_p + (t \lll 7)$
    if $v = 2$ **then**
      $x_p \leftarrow x_p \lll \lfloor t/2^{27} \rfloor$ ▷ the five high bits of t
    end if
    if $i > 0$ **and** $i \bmod M = 0$ **then**
      if $v = 2$ **then**
        $x \leftarrow \text{Perm1}(x, \text{true}, \pi_{i/M-1}, 5)$
      else
        $x \leftarrow \text{Perm1}(x, \text{true}, 0, 7)$
      end if
    end if
  end for
  return x
end function
function DecipherBlock($x_0, \ldots, x_{K-1}$; s, π, v)
  for $i \leftarrow RK - 1$ **down to** 0 **do**
    if $i > 0$ **and** $i \bmod M = 0$ **then**
      if $v = 2$ **then**
        $x \leftarrow \text{Perm1}(x, \text{false}, \pi_{i/M-1}, 5)$
      else
        $x \leftarrow \text{Perm1}(x, \text{false}, 0, 7)$
      end if
    end if
    $p \leftarrow i \bmod K$
    $t \leftarrow q(x_{(p-1) \bmod K} \oplus s_i)$
    if $v = 2$ **then**
      $x_p \leftarrow x_p \ggg \lfloor t/2^{27} \rfloor$ ▷ right rotation
    end if
    $x_p \leftarrow x_p - (t \lll 7)$
  end for
  return x
end function

Algorithm 4 Chaining blocks (CBC with a zero starting value)

function EncipherCBC($P_0, \ldots, P_{n-1}$; S) ▷ S: the schedule (s, π, v)
    $C_{-1} \leftarrow 0$
    for $b \leftarrow 0$ **to** $n-1$ **do**
        $C_b \leftarrow$ EncipherBlock($P_b \oplus C_{b-1}$; S)
    end for
    return $C_0, \ldots, C_{n-1}$
end function
function DecipherCBC($C_0, \ldots, C_{n-1}$; S)
    $C_{-1} \leftarrow 0$
    for $b \leftarrow 0$ **to** $n-1$ **do**
        $P_b \leftarrow$ DecipherBlock(C_b; S) $\oplus\, C_{b-1}$
    end for
    return $P_0, \ldots, P_{n-1}$
end function

Algorithm 5 Saving the password file (FPMQ3)

function Save(*passphrase*, *entries*)
    xml ← the entries written as XML
    salt ← 16 random bytes
    master ← PBKDF2-HMAC-SHA-256(*passphrase*, *salt*, 600000, 32)
    $k_c \leftarrow$ HKDF-Expand(*master*, "FPMQ3 sbc", 64)
    $k_m \leftarrow$ HKDF-Expand(*master*, "FPMQ3 mac", 32)
    $F \leftarrow$ 16 random bytes ‖ le32(length of *xml*) ‖ *xml*
    pad F with bytes FF_{16} to a multiple of 128 bytes
    $C \leftarrow$ EncipherCBC(F; KeySetupV2(k_c), $v = 2$)
    $H \leftarrow$ "FPMQ3\n" ‖ "kdf=pbkdf2-sha256; iters=600000; salt=" ‖ base64(*salt*) ‖ "\n"
    tag ← HMAC-SHA-256(k_m, $H \,\|\, C$)
    write H, then "mac=hmac-sha256; tag=" ‖ base64(*tag*), then base64(C) in lines of 64 characters
    erase *master*, k_c, k_m, F and *xml* from memory
end function

Algorithm 6 Opening the password file (FPMQ1, FPMQ2 or FPMQ3)

function OPEN(*file*, *passphrase*)
  read the version line, the salt, the repetition count c, the tag and C
  if the version is not FPMQ1, FPMQ2 or FPMQ3 **then**
    fail: corrupt file
  end if
  if $c \neq 600000$ **then** ▷ checked before any key is made
    fail: corrupt file
  end if
  if not FPMQ1 **and** the header differs from what SAVE writes **then**
    fail: corrupt file
  end if
  if FPMQ3 **then**
    derive k_c, k_m as in SAVE; $data \leftarrow H \,\|\, C$; *schedule* $\leftarrow$ KEYSETUPV2, $v = 2$
  else if FPMQ2 **then**
    the same, with the labels "FPMQ2 sbc" and "FPMQ2 mac"; *schedule* $\leftarrow$ KEYSETUPV1, $v = 1$
  else ▷ FPMQ1
    $d \leftarrow$ PBKDF2-HMAC-SHA-256(*passphrase*, *salt*, 600000, 96)
    $k_c \leftarrow d_{0\ldots63}$; $k_m \leftarrow d_{64\ldots95}$; $data \leftarrow C$; *schedule* $\leftarrow$ KEYSETUPV0, $v = 0$
  end if
  if HMAC-SHA-256(k_m, *data*) $\neq$ *tag* **then** ▷ compared in constant time
    fail: wrong passphrase, or the file was changed
  end if
  $F \leftarrow$ DECIPHERCBC(C; *schedule*(k_c))
  skip 16 bytes; read the length; **return** the entries parsed from the XML
end function

5 Remaining attacks and analyses

The review established the weaknesses above and measured some properties (Appendix A). It did not attempt the analyses below, which a cipher must survive before it can be trusted for important data.

5.1 Analyses of the cipher

1. **Strength of a single round group.** The slide attack pays only if 64 steps are much weaker than 768. That is usual, but it has not been shown for SBC. How much work recovers the key from one input and output of 64 steps?

   A single step, by contrast, is easily undone, because q is easy to invert. It is a T-function [9], and its derivative $q'(x) = 2nx + (an + c)$ is odd for every x. Once input bits 0 to $j-1$ are known, output bit j is input bit j plus a known quantity, so the input can be recovered from $y = q(x)$ bit by bit, from the lowest up, in 32 short steps. Newton's method is faster. It works modulo 2^{32} because $q'(x)$ is odd, and so has an inverse modulo 2^{32}:

   $$x \leftarrow x - \big(q(x) - y\big)\, q'(x)^{-1} \bmod 2^{32}. \tag{5.1}$$

   Each pass doubles the number of correct low bits, provided the starting value already has the right lowest bit. For $q(x) = (x+1)(2x+1)$ that bit is easy to find, because $q(x) \equiv x + 1 \pmod 2$: start from y with its lowest bit reversed. One correct bit then becomes 2, 4, 8, 16 and 32, so five passes give all 32. (Tested on 20,000 random values: five passes inverted every one; four inverted about half.) Rivest characterized exactly which polynomials are one-to-one modulo 2^w [10], and inverting them is routine. Neither RC6 nor SBC relies on its quadratic being hard to invert: decryption never inverts q, but only computes it again, forward.

   The consequence is that one step whose input and output are both known gives up its subkey at once. If x_p and x_{p-1} are the words before a version 0 or 1 step and x'_p the changed word after it, then

   $$t = (x'_p - x_p) \ggg 7, \tag{5.2}$$
   $$s_i = q^{-1}(t) \oplus x_{p-1}. \tag{5.3}$$

   In version 2 the rotation amount r is unknown. An attacker can try all 32 values, compute $t = \big((x'_p \ggg r) - x_p\big) \ggg 7$ for each, and keep those for which r equals the five high bits of t. The true value always survives, and each of the 31 wrong ones survives by chance with

probability 1/32, so on average about two candidates for the subkey remain. (Over 20,000 random steps the mean was 1.96, and the true subkey was always among them.) So in every version the protection lies not in any single step but in keeping the words between steps hidden. Within 64 steps each word is changed twice, and the question above is how much of that hiding survives.

2. **Differential cryptanalysis** [11]. Search for differences in the input that pass through a few rounds with high probability, and from them estimate how many rounds an attack could reach. The quadratic q is a *T-function* [9]: each output bit depends only on the input bits at the same place and below. Differences in the low bits therefore propagate predictably within q, and only the rotation and the addition carry them upward.

3. **Linear cryptanalysis** [12]. The two lowest output bits of q are linear: bit 0 is the complement of input bit 0, and bit 1 is a linear function of input bits 0 and 1. In version 0 of the 64-bit library a similar property made four bits of every step exactly linear. In the 32-bit cipher the addition in step (2.2) hides them behind carries. Whether useful linear approximations survive several rounds is not known.

4. **Reduced-round attacks.** Attack versions of SBC with 1, 2, 3, and more rounds, to learn how large the safety margin of 24 rounds actually is. This is the standard way of measuring a new cipher.

5. **The permutation.** In versions 0 and 1, `perm1` uses no key, and its choice depends on the exclusive-or of q applied to every word, which does not depend on their order. It reads only 30 of those 32 bits. It produces a great variety of rearrangements (65,535 different ones from 65,536 random blocks), so it mixes well, but anyone who knows the state at that point can compute it. Version 2 (Section 3.6) combines the driving number with a secret subkey for each permutation and reads all 32 bits. Questions remain for both forms. Does the permutation add strength, or only spreading? Are there blocks it leaves in a predictable order? The rearrangements all come from one family, eight moves of adding and folding, so even when keyed there are at most 2^{32} of them for each permutation, out of the 32!, or about 2^{117}, possible orders. Whether that family has structure an attacker can use is not known. A permutation drawn from all 32! orders with the key would remove the question, at the cost of more code.

6. **Algebraic attacks.** Each step is a polynomial of low degree over the integers modulo 2^{32}, mixed with exclusive-or. Methods that solve systems of such equations should be tried on reduced rounds.

7. **A rotation that depends on the data.** In versions 0 and 1 every step rotates by the same fixed amount, seven places. RC5 [13] and RC6 [14] instead rotate by an amount taken

from the data; in RC6 it is the five high bits of a quadratic of another word, and what is rotated is the word being changed. Version 2 of SBC does the same (Section 3.6): it rotates the changed word x_p by the five high bits of $q(x_{p-1} \oplus s_i)$, after the fixed rotation and the addition. An earlier idea, rotating the output of q by its own high bits, was set aside: there the amount is tied to the very value it moves, whereas the changed word holds independent data. The step remains reversible, because x_{p-1} and s_i are unchanged by it. The change helps in three ways.

First, it breaks the property that each output bit of q depends only on the input bits at the same place and below. The high bits of q depend on every input bit, and they decide where every bit of x_p lands, so within a single step the low bits of the result depend on the high bits of the input. In versions 0 and 1 that happens only through the fixed rotation, over several steps.

Second, it breaks the regular algebra of the step. A rotation by a varying amount is not a polynomial over the integers modulo 2^{32}, so the regularities of a quadratic, such as its constant second difference, $q(x + d_1 + d_2) - q(x + d_1) - q(x + d_2) + q(x) = 2n\, d_1 d_2$, no longer pass through the step intact. That makes the algebraic and higher-order differential attacks above harder.

Third, a differential or linear attack follows a chosen pattern of differences or bits through the steps. With a data-dependent rotation it must also predict the rotation amount at every step, one of 32 values, unless it can force those five bits. That multiplies its cost at every step.

Data-dependent rotations have weaknesses of their own: the best attacks on RC5 work by controlling the rotation amounts [15]. What remains is to learn whether an attacker can steer the five high bits of q often enough to matter, and to repeat for version 2 the differential, linear and reduced-round analyses of items 2 to 4.

8. **Statistical tests.** Run the output of SBC in counter mode through standard test batteries for random numbers, such as NIST SP 800-22 or PractRand, at full and at reduced rounds.

9. **Related keys under versions 1 and 2.** With subkeys from HKDF, no relation between keys is known to carry over to the subkeys. In `fpwdman` the keys come from PBKDF2 and cannot be chosen by an attacker, so this matters mainly for SBC as a standalone cipher.

5.2 Analyses of the file format and the program

10. **Old copies of the file.** A legacy copy of the file, under the same passphrase as the current one, lets an attacker guess that passphrase quickly and then open the current file.

Version 4.0 makes no backup copy, but copies can survive elsewhere: a `.bak` written by an earlier version, the user's own backups, synchronized folders, and snapshots kept by the operating system. After converting, the user should delete such copies where possible and then change the master passphrase, which makes any copy that cannot be found harmless to the current file.

11. **Memory-hard key derivation.** PBKDF2 can be computed cheaply in parallel on graphics processors. Argon2id [16], which also requires much memory, is now preferred [17]. A future format could use it.

12. **Unbounded inputs.** The program does not yet limit the size of a file it will read, nor the size a legacy file may claim for its decompressed contents. A crafted file could therefore demand a very large allocation. Limits should be set before any decoding.

13. **The length of the file.** Encryption hides the contents but not their approximate size, which reveals roughly how many entries the file holds. Padding to a fixed size would hide it.

14. **Timing.** The rotations, including version 2's data-dependent one, are single processor instructions whose time does not depend on the amount. But `perm1` writes words to places that depend on the data, and in version 2 on a subkey as well. On a shared computer, the timing of memory caches could in principle reveal something about those places, and so about the subkey. This matters only to an attacker running programs on the same machine at the same time.

15. **Secrets left in memory.** The program erases keys and plaintext after use, but Qt's strings are copied freely, so some copies cannot be reached and erased. The clipboard, the operating system's swap file and crash dumps can also hold secrets.

16. **Word size.** SBC could use 64-bit words, with a 64-bit q, in place of 32-bit ones. This would not make the password file noticeably safer. The practical attack on a stolen file is guessing the passphrase, which costs the same whatever the word size, and the key and block sizes, which would stay at 512 and 1024 bits, are already far beyond exhaustive search. Inside the cipher the change would cut both ways. Only the two lowest bits of q are linear, so the weak share of each word would fall from 2 bits in 32 to 2 in 64, and the longer carries of 64-bit addition and multiplication would mix the high bits more. But q moves information only upward within a word, and in versions 0 and 1 only the fixed rotation brings it down again: with a rotation of seven places, a high bit takes about nine steps to reach the bottom of a 64-bit word, against about five in a 32-bit word. (Version 2's data-dependent rotation narrows that difference.) A 1024-bit block would also hold 16 words instead of 32, and the regularities of the quadratic, such as its constant second

difference, would be unchanged. A 64-bit SBC would therefore have to be measured afresh, and it would be a new version of the cipher and of the file format.

17. **Comparison with standard ciphers.** SBC has had one reviewer. The standard ciphers, such as AES-256-GCM [18] and XChaCha20-Poly1305, have had decades of study by many. Two earlier reviews of `fpwdman` recommended writing new files with one of them and reading SBC files only. That remains the most conservative choice; the work in this article is the alternative of strengthening SBC and inviting its analysis.

Where further effort would count most. Version 2 has made two of the changes this list once recommended: the rotation that depends on the data (item 7) and the keyed permutation (item 5). In order of their benefit to the safety of the password file, the most useful remaining steps are these.

(a) Argon2id in place of PBKDF2 (item 11). It defends against the attack that matters most in practice, guessing the passphrase on many processors at once.

(b) Analysis of version 2: the differential, linear and reduced-round studies of items 2 to 4, repeated for the new step, and the open questions of items 5 and 7.

(c) A permutation drawn from all 32! orders with the key (item 5), if that analysis finds the add-and-fold family to be a weakness.

(d) A standard cipher for new files (item 17). It gives the greatest assurance, at the price of giving up SBC.

The first three keep SBC; the fourth replaces it. A change of word size is not on the list, for the reasons given above. For now the author keeps SBC, so as to continue its study.

A Measurements

All measurements below used keys and data generated at random for the purpose, or fixed test vectors. A Python implementation of SBC version 0 was first checked bit for bit against the compiled C++ program on a two-block test vector; the version 1 and version 2 test vectors in the program's tests were computed with a Python implementation written from the specification, using Python's own HMAC and SHA-256, which also reproduces the version 0 vector.

Spreading of changes. Mean number of the 1024 output bits changed when one bit of the input is changed, over 100 random keys and blocks (a random result would change 512):

Rounds	Mean bits changed	Fewest in any trial
1	249.2	14
2	512.2	481
3	511.9	476
4	513.4	470
24	513.6	475

Changing one bit of the key changed 510.5 bits on average after 24 rounds.

The same comparison for versions 1 and 2, over 2,000 random keys and blocks each, with the standard error of the mean:

Rounds	Version 1	Version 2
1	261.3 ± 3.3	266.5 ± 3.3
2	511.8 ± 0.4	511.0 ± 0.4

After one round the result depends strongly on which word was changed, since a change late in the ring passes through few steps; in some trials only 2 or 3 bits changed. Across the block, then, version 2 spreads a change no faster than version 1, which already matched a random result after two rounds. What version 2 changes is the behaviour within each word, below.

Within one step (version 2's rotation). For random x_p, x_{p-1} and s_i, one input bit of x_{p-1} was flipped and the bits of the new x_p that changed were counted, over 5,000 trials for each bit:

Input bit flipped	0	8	16	24	31
Version 1, bits changed	16.45	13.42	9.19	5.18	2.02
Version 2, bits changed	15.95	15.92	15.76	15.57	16.06

In version 1 a high input bit reaches few output bits, because each output bit of q depends only on the input bits at its own place and below. In version 2 every input bit changes about half of the 32. When bit 31 was flipped, over 20,000 trials, each of the output bits 0 to 5 changed with a frequency between 0.495 and 0.505 in version 2, and never in version 1.

Linearity of q. The algebraic degree of output bits 0 to 5 of q, as functions of the input bits at the same place and below, is 1, 1, 2, 3, 4, 5. Only the two lowest bits are linear.

The permutation. In versions 0 and 1, `perm1` reads 30 of the 32 bits of its combined value (bits 9 and 10 are never used); with the rotation by five of version 2 it reads all 32. Among 65,536 random values it produced 65,535 different rearrangements in one sample, and 65,536 in a second sample for each of the two rotations.

In version 2, one random block rearranged under two random permutation subkeys put on average 31.0 of its 32 words in different places, and in 2,000 trials the two orders were never the same. Two blocks that differed in a single bit, rearranged under the same subkey, never received the same order in 2,000 trials.

Timing of the rotation. In the optimized program, the version 2 rotation compiles to two instructions, a shift that extracts the amount (`shr ecx,1Bh`) and a rotation by it (`rol edx,cl`), with no branch.

Identical rounds in version 0. In all 10 checks, the state after permutation $m + 1$ equalled one fixed 64-step function applied to the state after permutation m. Under version 1, no two of the 24 rounds have the same subkeys, while under version 0 all 24 do; the program's tests check both.

Speed. One save and one open, in the program's debug build: 6.6 seconds for FPMQ1 and 2.3 seconds for FPMQ2. In a later run, made while other programs were being compiled, FPMQ1 took 9.0 seconds, FPMQ2 3.0 and FPMQ3 3.0; version 2 of the cipher adds no measurable time.

Tests. Version 4.1 passes 41 tests of the cipher and key derivation, including the published test vectors for SHA-256, HMAC-SHA-256, PBKDF2 and HKDF and known-answer vectors for all three cipher versions, and 26 tests of the file formats, including the conversion of legacy, FPMQ1 and FPMQ2 files to FPMQ3, the rejection of altered FPMQ3 files, and the rejection of FPMQ3 files relabelled as older versions.

References

[1] B. P. Wise, "Sliding block ciphers." Draft, 2006.

[2] K. Moriarty, B. Kaliski, and A. Rusch, "PKCS #5: Password-based cryptography specification version 2.1," RFC 8018, Internet Engineering Task Force, 2017.

[3] H. Krawczyk, M. Bellare, and R. Canetti, "HMAC: Keyed-hashing for message authentication," RFC 2104, Internet Engineering Task Force, 1997.

[4] National Institute of Standards and Technology, "Secure hash standard (SHS)," FIPS PUB 180-4, National Institute of Standards and Technology, 2015.

[5] H. Krawczyk and P. Eronen, "HMAC-based extract-and-expand key derivation function (HKDF)," RFC 5869, Internet Engineering Task Force, 2010.

[6] M. Bellare and C. Namprempre, "Authenticated encryption: Relations among notions and analysis of the generic composition paradigm," in *Advances in Cryptology – ASIACRYPT 2000*, vol. 1976 of *Lecture Notes in Computer Science*, Springer, 2000.

[7] A. Biryukov and D. Wagner, "Slide attacks," in *Fast Software Encryption (FSE '99)*, vol. 1636 of *Lecture Notes in Computer Science*, Springer, 1999.

[8] A. Biryukov and D. Wagner, "Advanced slide attacks," in *Advances in Cryptology – EUROCRYPT 2000*, vol. 1807 of *Lecture Notes in Computer Science*, Springer, 2000.

[9] A. Klimov and A. Shamir, "A new class of invertible mappings," in *Cryptographic Hardware and Embedded Systems (CHES 2002)*, vol. 2523 of *Lecture Notes in Computer Science*, Springer, 2002.

[10] R. L. Rivest, "Permutation polynomials modulo 2^w," *Finite Fields and Their Applications*, vol. 7, no. 2, 2001.

[11] E. Biham and A. Shamir, "Differential cryptanalysis of DES-like cryptosystems," *Journal of Cryptology*, vol. 4, no. 1, 1991.

[12] M. Matsui, "Linear cryptanalysis method for DES cipher," in *Advances in Cryptology – EUROCRYPT '93*, vol. 765 of *Lecture Notes in Computer Science*, Springer, 1994.

[13] R. L. Rivest, "The RC5 encryption algorithm," in *Fast Software Encryption (FSE '94)*, vol. 1008 of *Lecture Notes in Computer Science*, Springer, 1995.

[14] R. L. Rivest, M. J. B. Robshaw, R. Sidney, and Y. L. Yin, "The RC6 block cipher," in *First Advanced Encryption Standard (AES) Candidate Conference*, 1998.

[15] A. Biryukov and E. Kushilevitz, "Improved cryptanalysis of RC5," in *Advances in Cryptology – EUROCRYPT '98*, vol. 1403 of *Lecture Notes in Computer Science*, Springer, 1998.

[16] A. Biryukov, D. Dinu, D. Khovratovich, and S. Josefsson, "Argon2 memory-hard function for password hashing and proof-of-work applications," RFC 9106, Internet Engineering Task Force, 2021.

[17] OWASP Foundation, "Password storage cheat sheet." https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html.

[18] National Institute of Standards and Technology, "Advanced encryption standard (AES)," FIPS PUB 197, National Institute of Standards and Technology, 2001.